

Documentation

Team name: A4

Date: 01/07/2026

Team members

Andrey Sysoev

SamRB-dev

Md Mamun Hossain

Umut Karakaya

Introduction

The topic of this project is a Smart Room Monitoring System. The goal of the prototype is to monitor basic indoor room conditions by using sensors, wireless communication and a dashboard. The system measures temperature, humidity, light level and sound level. These values are sent from the sensor device to a Raspberry Pi, stored in a database and shown to the user through a web or mobile dashboard.

This project is related to the Internet of Things. IoT describes physical devices that are connected to a network and can collect and exchange data. In this project, the sensors collect information from the room, the Arduino/ESP32 sends this information wirelessly and the Raspberry Pi stores and processes the data.

The project is also related to Wireless Sensor Networks. A Wireless Sensor Network consists of sensor nodes that collect environmental data and transmit it wirelessly. In our prototype, the Arduino/ESP32 with connected sensors works as the sensor node. It sends the measured data over Wi-Fi by using MQTT.

The target application is indoor room monitoring for a student room, office, laboratory room or small smart home setup. The system helps the user understand whether the room is warm, humid, bright, dark, quiet or noisy. The current prototype focuses on measuring, transmitting, storing and displaying sensor data.

Concept description

The prototype follows this general data flow:

Sensors → Arduino/ESP32 → MQTT Broker → Raspberry Pi → SQLite Database → Dashboard

The sensors measure the room conditions. The Arduino/ESP32 reads the sensor values and publishes them as MQTT messages. The MQTT broker forwards the messages to the Raspberry Pi. The Raspberry Pi receives the data, processes it and stores it in a SQLite database. The dashboard shows the stored or latest sensor values to the user.

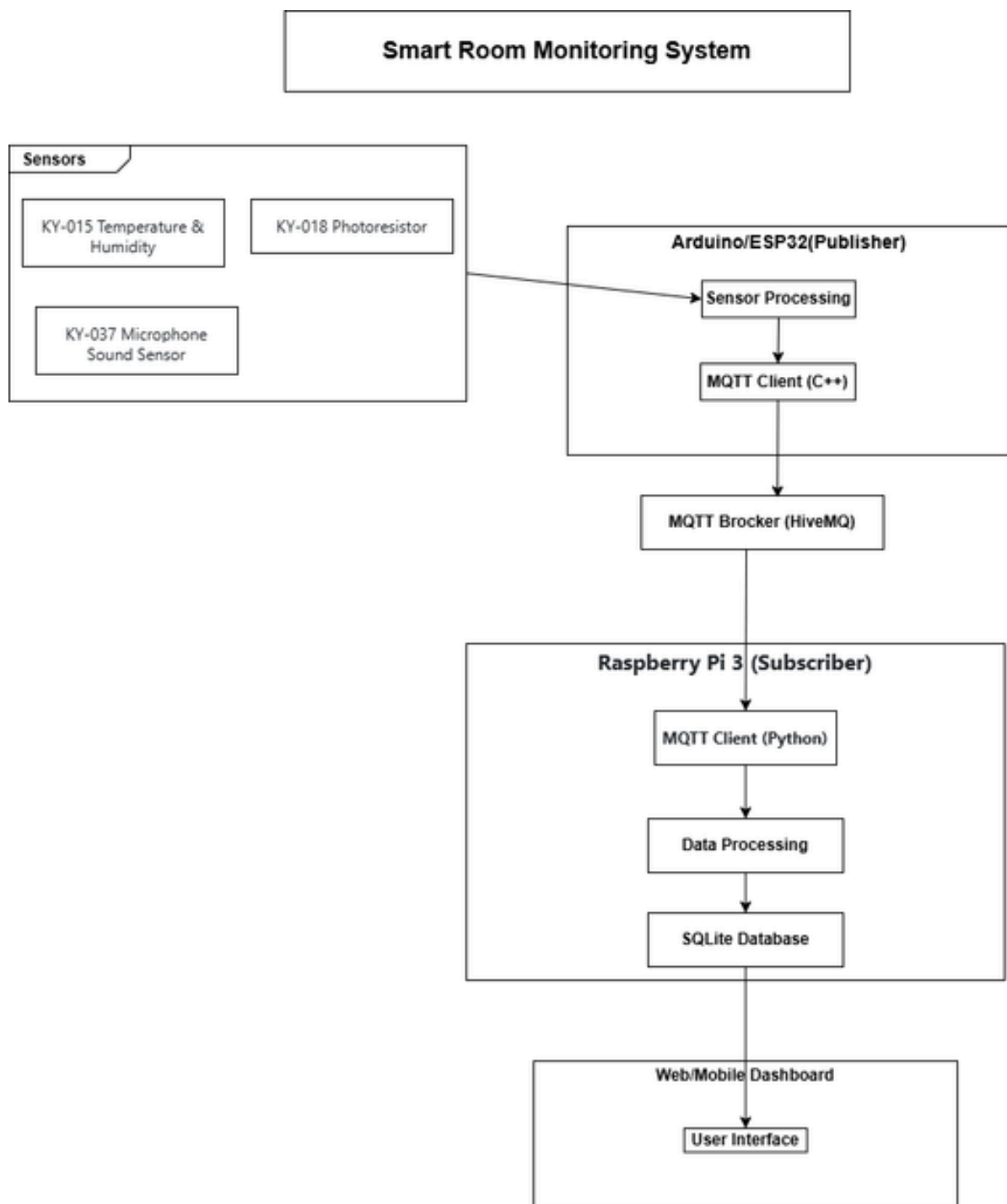


Figure 1: Block diagram of the Smart Room Monitoring System

The system uses three sensor components. The KY-015 sensor measures temperature and humidity. The KY-018 photoresistor measures the light level. The KY-037 microphone sensor measures the sound level. These sensors are connected to the Arduino/ESP32, which works as the MQTT publisher.

HiveMQ is used as the MQTT broker. The Raspberry Pi 3 works as the MQTT subscriber. It receives the sensor messages from the broker and stores them in the SQLite database. The dashboard is the user interface of the system. The user can view the room conditions without directly accessing the hardware, MQTT broker or database.

The block diagram shows the communication order. First, the sensors provide temperature, humidity, light and sound readings. Then the Arduino/ESP32 publishes MQTT messages. The broker forwards these messages to the Raspberry Pi. The Raspberry Pi inserts the values into the database. After that, the dashboard can request and display the latest sensor data.

The main functional requirements are room monitoring, sensor reading, wireless data transmission, data storage and dashboard display. The system shall read data from the KY-015, KY-018 and KY-037 sensors. It shall transmit the measured values from the Arduino/ESP32 to the Raspberry Pi by using MQTT. It shall store the received data in a SQLite database and provide an interface for the user.

The main non functional requirements are performance, availability, usability and extensibility. The sensors should be sampled regularly. The system should run continuously during a monitoring session. The dashboard should respond in an acceptable time. The system should also allow adding new sensors later without a major redesign.

Project/Team management

The project was divided into smaller parts to make the work easier to organize. The main parts were sensor setup, Arduino/ESP32 programming, MQTT communication, Raspberry Pi subscriber implementation, database handling, dashboard development, diagrams and documentation.

The team first defined the project idea and the system requirements. After that, the system architecture was planned with block diagrams, requirement diagrams, a sequence diagram and a use case diagram. These diagrams helped the team understand the data flow between the sensors, Arduino/ESP32, MQTT broker, Raspberry Pi, database and dashboard.

The implementation was done step by step. First, the sensors were connected and tested. Then the Arduino/ESP32 was prepared to publish sensor values through MQTT. After that, the Raspberry Pi subscriber was implemented in Python. The subscriber receives MQTT messages, reads the JSON payload and stores the data in the SQLite database. Finally, the dashboard was used to display the room monitoring values.

GitHub was used for code and file management. The team used an iterative method, where each module was tested separately before combining it with the full system.

Andrey Sysoev worked on: Arduino/ESP32 programming, MQTT communication, Raspberry Pi subscriber implementation, database handling, dashboard development

Sadim Rahman Badhan worked on : Arduino/ESP32 programming, MQTT communication, Raspberry Pi subscriber implementation, database handling, dashboard development

Md Mamun Hossain worked on: diagrams, documentation, MQTT communication

Umut Karakaya worked on: diagrams, documentation, MQTT communication

Technologies

The project uses technologies from embedded systems, IoT communication, database systems and user interface development. The main technologies are Arduino/ESP32, sensors, Wi-Fi, MQTT, HiveMQ, Raspberry Pi 3, Python, SQLite and a dashboard.

The sensor part consists of three modules. The KY-015 sensor is used for temperature and humidity measurement. The KY-018 photoresistor is used for light level measurement. The KY-037 microphone sensor is used for sound level measurement. These sensors provide the environmental data for the room monitoring system.

The Arduino/ESP32 works as the sensor node. It reads the connected sensors and sends the measured values through Wi-Fi. The embedded code is written in Arduino C/C++. The Arduino/ESP32 acts as the MQTT publisher.

MQTT is used as the communication protocol. It is a lightweight publish/subscribe protocol and is suitable for IoT systems. In this project, the Arduino/ESP32 publishes sensor data to MQTT topics. The Raspberry Pi subscribes to these topics and receives the messages.

HiveMQ is used as the MQTT broker. The broker forwards the messages from the publisher to the subscriber. This means that the Arduino/ESP32 and Raspberry Pi do not need to communicate directly with each other.

The Raspberry Pi 3 runs the Python subscriber. The Python program uses MQTT communication, reads incoming JSON messages and stores the sensor data in a SQLite database. The database stores the sensor readings with timestamps, so the values can be used later for display or analysis.

The dashboard is the user interface of the system. It displays temperature, humidity, light level and sound level in a readable way. This allows the user to monitor the room without dealing with MQTT messages or database entries.

The main programming languages and tools used in the project are Arduino C/C++, Python, SQLite/SQL, Bash scripts, MQTT, HiveMQ, GitHub and draw.io.

Implementation

Our Smart Room Monitoring System established with several hardware and software. We used several environmental sensors like KY-015 Temperature and Humidity Sensor, KY-018 Photoresistor and KY-037 Sound Sensor. These sensors are wired with Arduino Uno R4 WiFi and ESP32 and continuously measure the room's temperature, humidity, light intensity and sound levels.

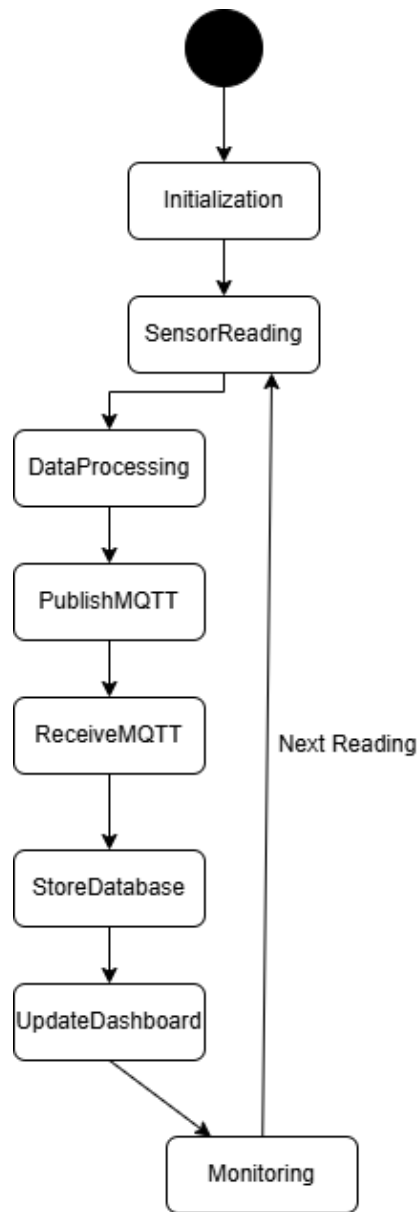


Figure 2: State Diagram

The Arduino Uno R4 WiFi and ESP32 process the sensor readings and converts them into JSON messages and publish the data to the MQTT broker.

Then we used a Raspberry Pi 3 Model B that works as the MQTT subscriber which subscribes to the MQTT topics and receives the published sensor data. It also stores the information in an SQLite database.

The SQLite database serves as the central storage of our system. It maintains current and historical data for real time monitoring and analysis.

Then we made a web dashboard that provides the graphical user interface of our system. It does the communication with the Raspberry Pi to receive sensor readings from the database and shows them in the user interface.

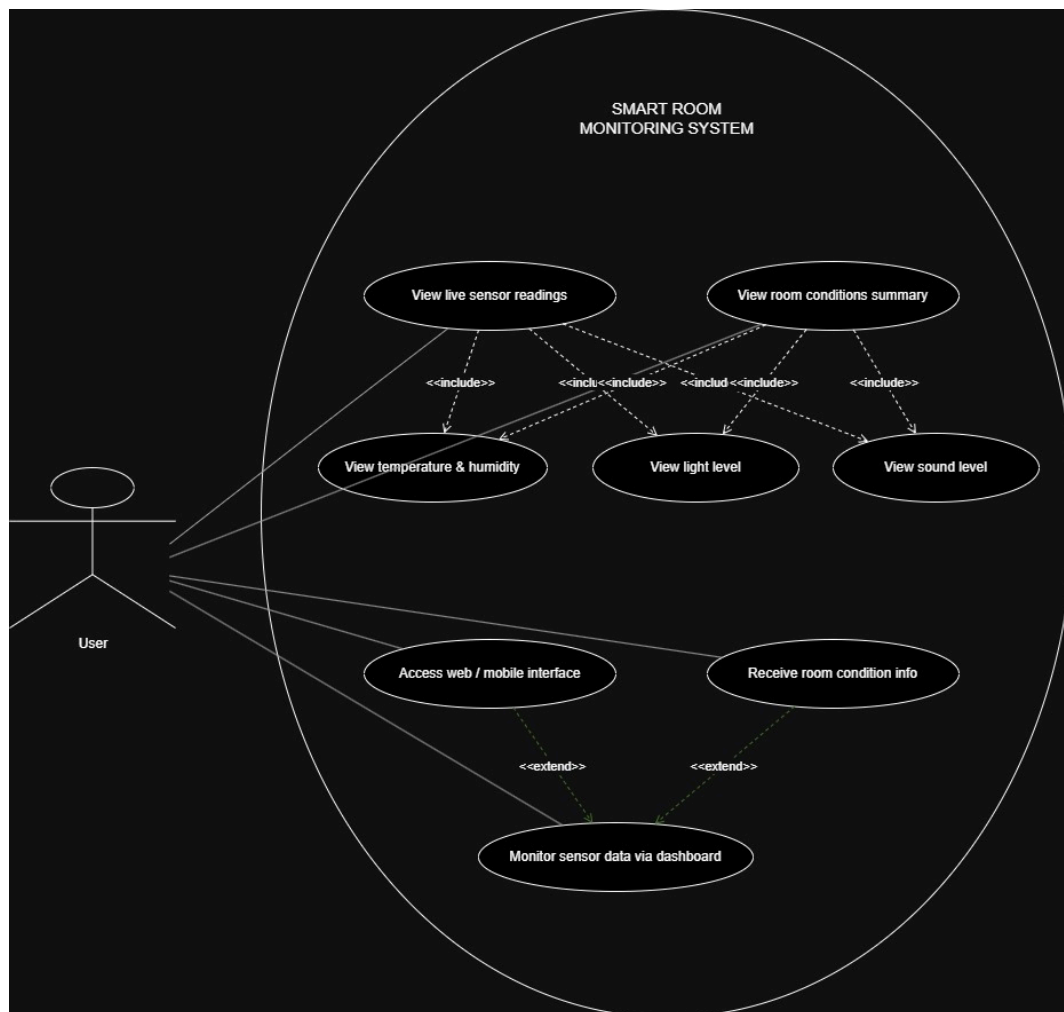


Figure 3: Use case Diagram

From the user perspective, the main use case is monitoring the room through the dashboard. The user can view live sensor readings, check temperature and humidity, check the light level and check the sound level. The user can also check the status of each sensor whether the sensor is active or offline.

Results

Our Smart Room Monitoring System was successfully implemented and tested using an Arduino Uno R4 WiFi, an ESP32, a Raspberry Pi 3 Model B, a KY-015 Temperature and Humidity Sensor, a KY-018 Photoresistor and a KY-037 Sound Sensor. At the time of testing, our system successfully measured temperature, humidity, light intensity and sound level and transmitted the readings to the Raspberry Pi through the MQTT protocol. The Raspberry Pi received the data and stored it in an SQLite database. A web dashboard displayed current temperature, humidity, light intensity and sound level, enabling users to monitor room conditions in real time.

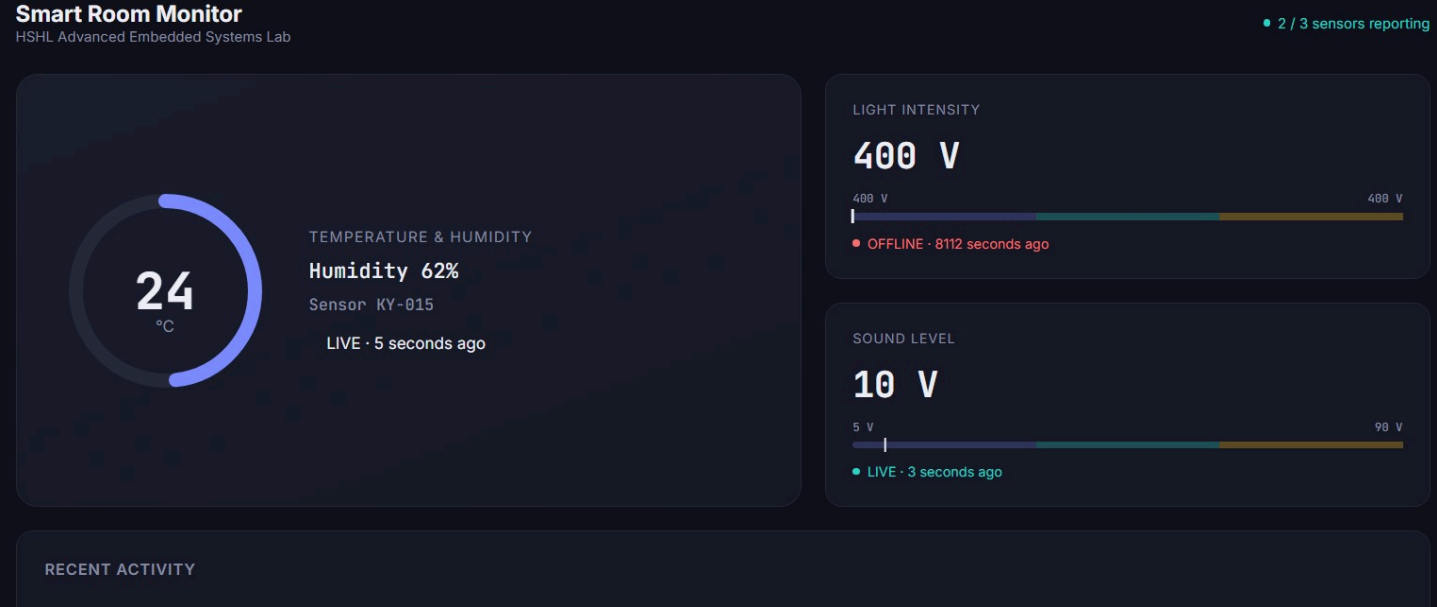


Figure 4: User Interface

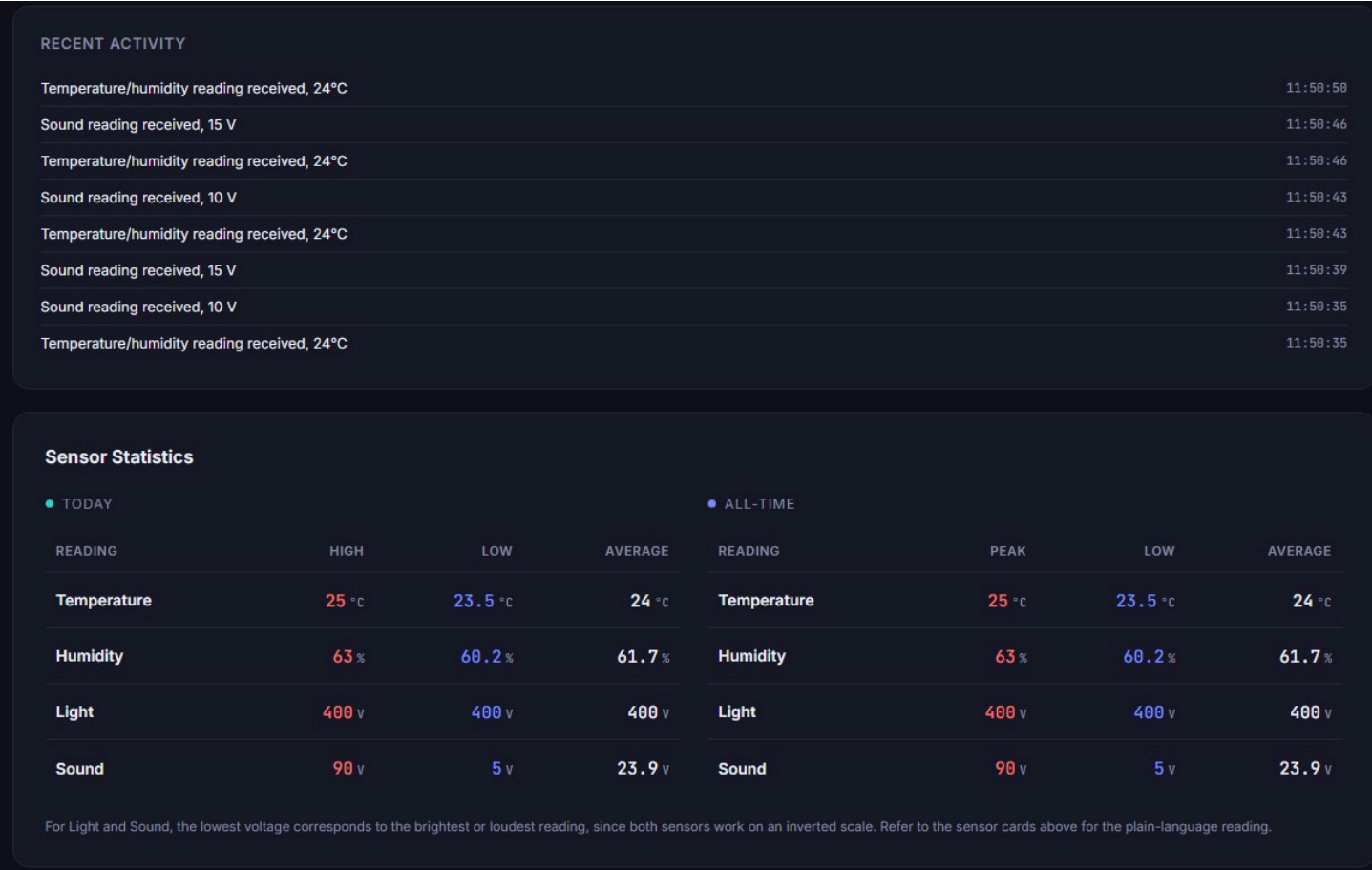


Figure 5: Serial Monitor

Figure 4 and 5 show the main dashboard of the system. The dashboard displays the latest sensor readings and indicates whether each sensor is currently online.

The temperature was measured at 24°C.

The humidity was 62%.

The light sensor reported a value of 400 V .

The sound sensor reported a value 10 V.

The dashboard also displays the status of each sensor. At the time of screenshot the temperature and sound sensors were actively sending data, while the light sensor temporarily appeared as offline. This status helps to identify error, communication failure or disconnected sensors.

Sources/References

[1]  *Implementing MQTT in Python with Code Examples*

[2]  *MQTT in Python with Paho Client: Beginner's Guide 2026*

[3] <https://eclipse.dev/paho/files/paho.mqtt.python/html/client.html#paho.mqtt.client.Client.connect>

[4]  *Securing MQTT Systems - MQTT Security Fundamentals*

[5] <https://sensorkit.joy-it.net/en/sensors/ky-015>

[6] <https://sensorkit.joy-it.net/en/sensors/ky-018>

[7] <https://sensorkit.joy-it.net/en/sensors/ky-037>